NAT Signal-Extraction Build Specification

Pattern Discovery, Feature Wiring, the Process Framework,
the ML Plan, and Cloud Deployment

NAT Quantitative Research Platform

Consolidated 2026-06-24

Contents

1 Scope and Source Inventory

This document consolidates the engineering specifications of the NAT platform's build/implementation track into a single reference. It is organized by subsystem rather than by source file. The source documents are:

- **Convolver discovery pipeline** — `convolver_implementation/01..14` (14 per-stage specs) plus the draft `convolver_implementation.txt`. See §2.
- **Feature wiring** — `nan_wiring/01..05` (whale-flow, position-tracker, feature wiring, wallet discovery, concentration viability). See §3.
- **Process framework** — `process_concept.md`, `process_signal_design.md`, `process_mi_targets_derivat` See §4.
- **ML implementation plan** — `ml_implementation_plan.txt`. See §5.
- **Cloud deployment and operations** — `cloud_deployment/0_overview.md`, `cloud_deployment/2_observ` two copies of `HETZNER_DEPLOYMENT_PLAN.md`, and `packaging/README.md`. See §6.

2 Convolver Pattern-Discovery Pipeline

The convolver discovers data-driven pattern kernels from OHLCV candle data via event-aligned SVD, then deploys them as an online matched filter on the 100 ms tick stream. One discovery run is executed per *(symbol, candle_freq)* pair, processing all event types and channels in a single pass.

2.1 Data ingestion and overall flow

Historical OHLCV is fetched from Hyperliquid's native `candleSnapshot` REST API (`scripts/data/fetch_candles` `POST https://api.hyperliquid.xyz/info`), using only the standard library (no CCXT). Pagination is 5000 candles/request ($\sim$3.5 days at 1m), auto-paginated and incremental. Minimum data: 50K candles ($\sim$35 days) for sample size, 6+ months for regime coverage. The offline pipeline is:

```
[0] fetch_candles      Hyperliquid candleSnapshot API -> parquet
[1] OHLCV decomposition    4 channels: body, upper_wick, lower_wick, volume
[2] event detection        6 boolean masks (breakout/turtle/trap x bull/bear)
[3] ATR normalization      center by mean, scale by ATR (shape-preserving)
[4] matrix assembly        stack W-candle windows around events -> X in R^{n x W}
[5] thin SVD               X = U S V^T; right singular vectors V_k are shapes
[6] IC gate                keep components whose loadings predict forward returns
[7] BH-FDR correction      control false discovery rate across all tests
[8] kernel library output  .npz (arrays) + .json (metadata)
```

Supporting stages run during discovery but outside the core linear flow: walk-forward validation, rolling SVD stability, and analytical-basis alignment. Online scoring (§2.12) is the separate production runtime. The key files are `scripts/analysis/convolver_discovery.py` (offline discovery, stages 0–8 plus validation), `scripts/algorithms/convolver_kernels.py` (shared math, data structures, persistence), and `scripts/algorithms/convolver.py` (online scoring).

Anti-overfitting architecture. Three independent safeguards each defend a distinct failure mode:

Layer	Mechanism	Failure mode defended
SVD	Return-blind decomposition	Data snooping
BH-FDR	Multiple-testing correction, $q = 0.05$	False discovery
Walk-forward	60/40 OOS IC validation	In-sample overfitting

2.2 Stage 1 — Candle decomposition

Raw OHLCV is decomposed into four orthogonal channels (`convolver_kernels.py:102-124`, `decompose_candles`)

Channel	Formula	Captures	Sign
`body`	$C - O$	Direction / momentum	signed (+ bullish)
`upper_wick`	$H - \max(C, O)$	Upside rejection	≥ 0
`lower_wick`	$\min(C, O) - L$	Downside rejection	≥ 0
`volume`	V	Participation	≥ 0

Body isolates direction from level noise; wicks capture rejection independently of body direction; volume is structurally independent of price. (BTC discovery: the trap-bull volume channel had EVR = 26.5%, SI = 2.74 — a stereotyped, distinct volume pattern.)

2.3 Stage 2 — ATR computation and window normalization

Average True Range (Wilder 1978), `compute_atr(H,L,C,period=14)` (`convolver_kernels.py:132-151`):

$$\mathrm{TR}_t = \max\big(H_t - L_t,\ |H_t - C_{t-1}|,\ |L_t - C_{t-1}|\big), \qquad \mathrm{ATR}_t = \frac{1}{N}\sum_{j=0}^{N-1} \mathrm{TR}_{t-j}. \tag{1}$$

The first `period` values are NaN (warmup). Each window is then shape-preservingly normalized (`normalize_window`, `convolver_kernels.py:159-168`):

$$x_{\mathrm{norm}} = \frac{x - \mathrm{mean}(x)}{\mathrm{ATR}_t}, \tag{2}$$

removing absolute level (mean-centering) and volatility scale (ATR division). Edge case: ATR ≤ 0 or non-finite returns the zero vector, which is then excluded by the validity check in matrix assembly.

2.4 Stage 3 — Event detection (six types)

Six boolean event masks are computed. Default parameters: Donchian lookback $N = 20$, volume multiplier `vol_mult`$= 1.5$, trap lookahead $K = 3$, ATR reversal threshold $\alpha = 1.0$. Let med denote the rolling median.

Breakouts (`convolver_discovery.py:80-97`) — Donchian breaks with volume confirmation:

$$\mathrm{breakout\_bull}_t:\ C_t > \max(H_{t-N..t-1}) \ \wedge\ V_t > \mathrm{vol\_mult} \cdot \mathrm{med}(V_{t-N..t-1}), \tag{3}$$

$$\mathrm{breakout\_bear}_t:\ C_t < \min(L_{t-N..t-1}) \ \wedge\ V_t > \mathrm{vol\_mult} \cdot \mathrm{med}(V_{t-N..t-1}). \tag{4}$$

Empirically the *least* informative type: 0 of 158 breakout candidates survived BH-FDR (their content is arbitraged away).

Turtle soup (`convolver_discovery.py:100-112`) — false range breaks, with *no* volume filter (false breaks are characterized by low follow-through):

$$\mathrm{turtle\_bull}_t:\ L_t < \min(L_{t-N..t-1}) \ \wedge\ C_t > \min(L_{t-N..t-1}), \tag{5}$$

$$\mathrm{turtle\_bear}_t:\ H_t > \max(H_{t-N..t-1}) \ \wedge\ C_t < \max(H_{t-N..t-1}). \tag{6}$$

Traps (`convolver_discovery.py:116-143`) — confirmed false breakouts: a volume-confirmed breakout that reverses by more than $\alpha \cdot \mathrm{ATR}$ within K bars. The *only* event type with a lookahead (K bars):

$$\mathrm{bull\_trap}_t:\ \mathrm{breakout\_bull}_t \ \wedge\ C_{t+K} < C_t - \alpha \cdot \mathrm{ATR}_t, \tag{7}$$

$$\mathrm{bear\_trap}_t:\ \mathrm{breakout\_bear}_t \ \wedge\ C_{t+K} > C_t + \alpha \cdot \mathrm{ATR}_t. \tag{8}$$

Traps carry the strongest forward information (highest rolling SVD stability) but are the *sample-size bottleneck* (11–14 events per 1000 candles).

2.5 Stage 4 — Event-aligned matrix assembly

For each (event type, channel) pair, all valid normalized W-candle windows ($W = 20$ default) ending at events are stacked into $X \in \mathbb{R}^{n \times W}$ (`build_event_matrix`, `convolver_discovery.py:167-207`). Three validity filters: event index $\geq W - 1$ (sufficient lookback); finite positive ATR at the event bar; no NaN in the window. Each run produces 6 event types $\times$ 4 channels = 24 matrices.

2.6 Stage 5 — Thin SVD, EVR truncation, stereotypy

Return-blind decomposition $X = USV^\top$ via `np.linalg.svd(X, full\_matrices=False)` (LAPACK `dgesdd`, $O(nW^2)$): $U \in \mathbb{R}^{n \times K}$ are per-event loadings, S the singular values, and the right singular vectors $V \in \mathbb{R}^{W \times K}$ are the *discovered shapes*. Components are kept up to the explained-variance-ratio threshold:

$$\mathrm{EVR}_k = \frac{\sigma_k^2}{\sum_j \sigma_j^2}, \qquad K = \min\Big\{k : \textstyle\sum_{j=1}^{k} \mathrm{EVR}_j \geq 0.80\Big\}. \tag{9}$$

The stereotypy index $\mathrm{SI} = \sigma_1/\sigma_2$ measures shape consistency (SI > 3: highly stereotyped). Crucially, SVD never sees forward returns — the IC gate alone connects shapes to predictability, preventing data snooping.

2.7 Stage 6 — Information-coefficient gate

For each surviving component, correlate its scaled per-event loadings with forward returns (`\_compute\_ic`, `convolver_discovery.py:215-230`):

$$\mathrm{IC}_k = \mathrm{corr}\big(U_k\,\sigma_k,\ r_{\text{forward}}\big), \qquad r_{\text{forward}}[t] = \log\frac{C[t+h]}{C[t]}. \tag{10}$$

The discovery horizon is the primary horizon only (`horizons[0]`, default $h = 10$ candles = 10 min at 60s). Components with $|\mathrm{IC}| \geq 0.03$ are retained (deliberately lenient — BH-FDR carries the multiple-testing burden). Significance uses a t-test under $H_0 : \mathrm{IC} = 0$:

$$t = \mathrm{IC}\sqrt{\frac{n-2}{1-\mathrm{IC}^2}}, \qquad p = 2\big(1 - F_t(|t|,\, n-2)\big), \tag{11}$$

via `scipy.stats.t.cdf` (the small-sample correction matters for sparse traps, $n < 100$). Samples with $n < 30$ return IC $= 0$, $p = 1$.

2.8 Stage 7 — Benjamini–Hochberg FDR correction

All p-values across every (event type, channel, component, horizon) test ($\sim$120 tests) are corrected as a batch (`\_bh\_fdr`, `convolver_discovery.py:233-251`), controlling the expected false-discovery proportion at $q = 0.05$: sort $p_{(1)} \leq \cdots \leq p_{(m)}$, find the largest rank j with $p_{(j)} \leq jq/m$, and reject (keep) all hypotheses of rank $\leq j$. Empirically: 158 IC-passing candidates $\to$ 6 survivors (all turtle/trap; zero breakouts).

2.9 Stage 8 — Kernel library persistence

Surviving kernels are saved in dual format (`convolver_kernels.py:348-455`): a compressed `.npz` ($n_{\text{kernels}} \times W$ array, $\sim 2\,\text{KB}$) plus a `.json` of metadata. The kernel record is:

```
@dataclass(frozen=True)
class ConvolverKernel:
    event_type: str       # "breakout_bull", "turtle_bear", ...
    channel: str          # "body" | "upper_wick" | "lower_wick" | "volume"
    component_idx: int    # SVD component index k (0-based)
    kernel: np.ndarray    # v_k in R^W, unit norm
    ic: float             # signed IC at primary horizon
    ic_pvalue: float      # t-test p-value
    evr: float            # explained variance ratio
```

`.npz`+`.json` is chosen over HDF5 (C dependency), Parquet (schema overhead), and pickle (code-execution risk).

2.10 Analytical basis dictionary and fallback kernels

A fixed set of 10 unit-norm parametric functions on $\tau \in [0, 1]$, discretized to W points (`analytical_basis(W, a=5.0)`, `convolver_kernels.py:233-277`), serves as an interpretability diagnostic and a fallback library:

Family	Name / formula	Shape
Polynomial	$\phi_{q1} = \tau^2$, $\phi_{q2} = (\tau - 0.5)^2$, $\phi_{q3} = (1-\tau)^2$	ramps / U-shape
Cubic	$\phi_{c1} = \tau^3$, $\phi_{c2} = (\tau - 0.5)^3$, $\phi_{c3} = (1-\tau)^3$, $\phi_{s1} = 3\tau^2 - 2\tau^3$	S-curves / smoothstep
Exponential	$\phi_{e1} = e^{-a\tau}$, $\phi_{e2} = e^{-a(1-\tau)}$, $\phi_{b1} = e^{-a(\tau-0.5)^2}$	spikes / Gaussian bump

Decay rate $a = 5.0$ (configurable in $[3, 8]$). Alignment is $\text{align}(v, \phi) = |\cos(v, \phi)|$; > 0.85 indicates a known analytical form, < 0.50 a novel data-driven shape. Six fallback kernels (body channel, one per event type) are used when no SVD library exists: breakout $= \pm\phi_{s1}$, turtle $= \pm\phi_{c2}$, trap $= \pm(\phi_{e2} - \phi_{e1})$.

2.11 Walk-forward validation and rolling stability

Two diagnostics gate the BH-FDR survivors. **Walk-forward** splits temporally (60% train / 40% test) and reports the decay ratio $\text{IC}_{\text{decay}} = \text{IC}_{\text{OOS}}/\text{IC}_{\text{IS}}$, with threshold ≥ 0.50. **Rolling SVD stability** measures the mean cosine similarity of the first right singular vector across consecutive rolling windows, ρ, with threshold ≥ 0.70. The joint interpretation:

IC decay	Stability ρ	Interpretation
≥ 0.50	≥ 0.70	Robust kernel — deploy
< 0.50	≥ 0.70	Shape real, needs more data for IC stability
≥ 0.50	< 0.70	Accidental IC, unstable shape
< 0.50	< 0.70	No signal

BTC (May 2026) sits in row 2: 0/6 passed walk-forward, but turtle-bull $\rho = 0.920$ and trap-bull $\rho = 0.941$ — shapes are real; only 12K candles (single regime) limit IC estimation.

2.12 Online cosine-similarity scoring

The production algorithm (`convolver.py`, class `Convolver(MicrostructureAlgorithm)`) is a matched filter (Turin 1960). It aggregates 600 ticks into a 60s micro-candle, decomposes into the four channels, normalizes the sliding W-window by ATR, and scores via cosine similarity. For unit-norm kernels, $s = \langle x, k \rangle / \|x\|$ (returns 0 if $\|x\| < 10^{-12}$). Multi-channel aggregation per event type e:

$$S^{(e)}(t) = \sum_c w_c \sum_j \mathrm{IC}_j \, s_j^{(e,c)}(t), \tag{12}$$

with channel weights $w_c = 0.25$ (equal) and IC weighting across kernels j. Eight features are emitted: six per-event similarities `alg_conv_breakout_bull/bear`, `alg_conv_turtle_bull/bear`, `alg_conv_trap_bull/bear` (range $[-1, 1]$); `alg_conv_best_score` $= \max|\cdot|$ over the six ($[0, 1]$); and `alg_conv_best_pattern` (index 0–5). Required columns: `raw_midprice`, `flow_volume_1s`. Warmup $= (W + \text{atr\_period}) \times \text{candle\_ticks} = (20 + 14) \times 600 = 20{,}400$ ticks ($\sim$34 min), NaN-blanked.

2.13 Operational integration and discovery roadmap

The convolver runs offline in Python (not in the Rust ingestor): the paper trader loads tick parquet, calls `convolver.run_batch`, appends the eight `alg_conv_*` columns, aggregates to bars, and z-scores to P80/P20 entry thresholds. Five operational steps refine and integrate it.

Step 0 — bar-aggregation fix. `alg_conv_best_score` is 0 for $\sim$95% of candles and spikes to 0.3–0.8 on a match. Aggregating five 60s candle scores into a 5-min bar with `bar_agg="mean"` dilutes a lone 0.8 to 0.16 — below the P80 entry threshold, producing zero trades (the June-1 gauntlet bug). The fix is `bar_agg="max"`, which preserves the peak match.

Step 1 — re-run BTC discovery. The deployed kernels date from 12,059 candles ($\sim$8.4 days); $\sim$30 days ($\sim$43K candles at 60s) is $\sim$3.6$\times$ more, projected to yield $\sim$550 trap / $\sim$1050 turtle / $\sim$1850 breakout events ($\sim$140 trap events/channel — above the 100 SVD threshold; $\sim$220 OOS traps). If re-discovered ICs rise the shapes are real; if they fall, the May-24 set was overfit. Back up before `-save` overwrites.

Step 2 — cross-symbol universality. Per-symbol discovery on ETH/SOL, then cosine similarity of matching kernels: > 0.80 means universal — pool BTC+ETH+SOL to $\sim$129K candles ($\sim$410 traps/channel) under one shared library; < 0.50 means symbol-specific libraries. Pooled discovery is a $\sim$50-LOC wrapper calling the SVD/IC/FDR functions directly.

Step 3 — convolver as ML features. The eight outputs are decorrelated from all microstructure features ($|\rho| < 0.2$, a different data representation), making them high-value inputs to the ML algorithms (§5) rather than a standalone strategy — the convolver need not pass the gauntlet alone if its features improve the models that consume them. This requires runner support for multi-algorithm chaining (`run_chain`: run the convolver first, `concat` its outputs, then run the ML algorithm on the augmented frame) — the same Wave-0 prerequisite as the bar-level fix. Drop from the ML set if LightGBM importance < 0.01.

Step 4 — quarterly re-discovery. Every 90 days ($\sim$130K new candles/symbol), re-run discovery, compare to the prior library by cosine similarity (> 0.85: stable, adopt; < 0.85: investigate a regime shift), and track FDR-survivor count, mean IC, decay ratio, and ρ over time — a data-volume learning curve revealing when discovery becomes OOS-robust. Cost is $O(n_{\text{candles}} \cdot W \cdot n_{\text{kernels}}) \approx 0.5$ s per day of data; warmup is $(W + \text{atr\_period}) \times 600 = 20{,}400$ ticks ($\sim$34 min, $\sim$2.4% of a day).

3 Feature Wiring: Whale Flow, Liquidation Risk, Concentration

Of the 236 schema features, 48 are uniformly NaN because the position/trade-attribution pipeline is unwired. The Rust computation already exists; what is missing is the data feed. This five-step plan unlocks up to 40 features (12 whale-flow + 13 liquidation-risk + 15 concentration). **Verified locally (Jun-16):** the wired binary populates all 31 whale/liquidation/concentration columns $\sim$6 min after start, and `WsTrade.users` is populated (wallet discovery works).

3.1 Step 01 — Whale flow from trade classification

A zero-cost heuristic: classify trades with notional $\geq$ \$100K as whale activity and feed them to the already-implemented `WhaleFlowBuffer` (1109 LOC in `ing-features/src/whale_flow.rs`). On each `WsMessage::Trades`, if price $\times$ size $\geq$ whale_threshold_usd, a `WhalePositionChange` is added (signed by trade side; wallet = `trade\_<tid>`; `is_market_maker=false`). Config: `[features.whale\_flow]` `whale\_threshold\_usd = 100000.0`. Unlocks 12 features: `whale_net_flow_1h/4h/24h`, `whale_flow_normalize` `whale_flow_momentum`, `whale_flow_intensity`, `whale_flow_roc`, `whale_buy_ratio`, `whale_directional_agreem` `active_whale_count`, `whale_total_activity`. Limitation: trade size $\neq$ position change; upgraded in Step 03.

3.2 Step 02 — Position tracker config and shared state

The fully-implemented `PositionTracker` (367 LOC, `ing/src/positions/tracker.rs`; polls public `clearinghouse_state(wallet)` REST, no auth) is given a config section and a thread-safe `SharedPositionState` (new `positions/state.rs`) bridging the tracker task to per-symbol `MarketState`. Config defaults: `enabled=false` (opt-in), `poll_interval_secs=60`, `max_concurrent_requests=10`, `discover_from_trades=true`, `max_tracked_wallets=50`, optional `initial_wallets`. `SharedPositionState` (built on `parking_lot::RwLock`) exposes `update`, `drain_deltas`, `get_liquidation_positions`, `get_concentration_positions`, and `snapshot_count`; `PositionSnapshot` already carries `liquidation_price`, `position_value`, `size`, `entry_price`.

3.3 Step 03 — Spawn tracker and wire 40 features

`PositionTracker` is spawned as a tokio task in `main.rs` (when `position_tracker.enabled`), feeding snapshots/deltas through `mpsc` channels to a consumer that updates `SharedPositionState`; an `Arc<SharedPositionState>` is passed to each symbol ingestor. In `compute_features()`, real position deltas *upgrade* whale-flow, and `liquidation::compute(...)` (13 features) and the `ConcentrationBuffer` (15 features) fire when positions are present. Data flow:

```
PositionTracker::poll_all()
  -> clearinghouse_state(wallet) per wallet
```

```
-> PositionSnapshot + PositionDelta
-> mpsc channels -> consumer -> SharedPositionState
-> MarketState::compute_features() reads SharedPositionState
-> features.whale_flow / liquidation_risk / concentration = Some(...)
-> Parquet writer (real values instead of NaN)
```

Graceful shutdown is automatic (the tracker is a read-only polling loop).

3.4 Step 04 — Wallet discovery from the trade stream

Whale addresses are discovered from `WsTrade.users: Option<(String,String)>` (maker/taker). A `WalletDiscovery` accumulator counts addresses; every 5 minutes the top-N new wallets are promoted to the tracker (capped at `max_tracked_wallets`). **Risk:** `WsTrade.users` may not be populated by the public WebSocket — a one-time diagnostic logs `Some` vs `None` on first trade; the fallback is curated `initial_wallets`, and Step-01 whale flow works regardless. (Field confirmed populated Jun-16.)

3.5 Step 05 — Concentration viability gate

A *decision gate*, not code: after 48h with the tracker live, concentration features (Herfindahl, Gini, top-N share) are judged against tracked-wallet count and open-interest coverage:

Wallets tracked	OI coverage	Verdict
50+	$> 20\%$	viable — no change
20–50	5–20%	noisy — add a `FEATURES.md` disclaimer
< 20	$< 5\%$	unavailable — keep NaN, documented

If not viable, the recommended action is Option A (keep NaN: no schema change, all downstream NaN-safe) over Option B (remove features: `count_all()` 236 $\rightarrow$ 221, a breaking schema change). Sanity ranges once populated: top5 0.05–0.50, HHI 0.01–0.25, Gini 0.30–0.80.

4 The Process Framework — NAT's Third Citizen

4.1 Concept and contract

A *process* is the third first-class citizen alongside features and algorithms:

Citizen	Question it answers	Contract
feature	what is computed	ingestor / derived columns
algorithm	how it trades	`MicrostructureAlgorithm` (`scripts/algorithms/`)
process	whether/where information about price action exists	`Process` (`scripts/processes/`)

Two kinds share one registry/CLI/persistence surface (`scripts/processes/base.py`): `EvaluationProcess` (`evaluate(df, ctx) -> ProcessResult`, scores existing columns) and `TransformProcess` (`transform(df, ctx) -> (derived_df, ProcessResult)`, produces new series such as PCA components or triple-barrier labels that the runner chains into any evaluation via `-score-with`). Conventions mirror algorithms: `@register` decorator (registry key = `name()` = config section); `required_columns(available)`

receives the schema for column pattern-selection; `data_level` is `"bars"` (default) or `"ticks"`; `describe()` returns a machine-readable param spec. **NaN policy (K2):** every process routes columns through `partition_usable_columns()` — dead, constant, or thin columns are skipped with a recorded reason (`all_nan`, `constant`, `n_valid<min`), never crashing a run.

4.2 Implemented processes

Name	Kind	Description
`ic_horizon`	evaluation	Expanding Spearman IC × horizon sweep; overlap-corrected $n_{\text{eff}} = n/h$; BH-FDR.
`mi_ksg`	evaluation	KSG mutual information vs returns; gate MI $\geq I_{\min}(\text{fee}, \sigma_r, \text{kurt})$; rank inputs.
`transfer_entropy`	evaluation	Directed TE(feature → 1-bar return), linear (default) or KSG, with reverse control.
`spectral`	evaluation	Tick PSD / Hurst / OU half-life / band IC (wraps spannung).
`ml_importance`	evaluation	LightGBM expanding walk-forward gain importance + net-PnL-after-cost.
`triple_barrier`	transform	López de Prado 3-barrier labels (`tb_label/tb_ret/tb_hit_bars`), vol-scaled.
`pca_combo`	transform	Train-prefix PCA → orthogonal `pc_1..pc_k`; holdout-only IC.

A note on `mi_ksg`: rank/copula-transforming inputs is mandatory — raw-scale KSG manufactures a spurious $\sim$0.07-bit noise floor when feature and return scales differ (verified on planted data). `ic_horizon` deliberately uses full-sample ρ with overlap-corrected n_{eff}, *not* the screener's expansion-point t-test, which treats overlapping windows as replications and flags noise.

Result schema and persistence. `ProcessResult` (schema_version 1) records `run_id`, process, kind, symbol, timeframe, params, data fingerprint, provenance (`git_sha`, dirty, timestamp), `features_tested/skipped`, `findings` (feature, horizon, metric, value, threshold, p, p_{adjusted}, `informative`), optional `derived`, and a `summary`. The authoritative JSON is written atomically to `data/research/processes/{ru` a queryable index row goes to the `process_results` table in `nat.db` (best-effort). **FDR scope:** BH is *per-run*; the index is a research log, not a meta-gate — gates G1/G4 remain authoritative for promotion. CLI: `nat process list/run/results/show`; tests via `nat test process` (48 planted-signal + no-lookahead + persistence tests).

4.3 Process set design (S1–S9): extracting uncorrelated signals

The design operationalizes one law — the information ratio budget:

$$\text{IR} \approx \overline{\text{IC}}\,\sqrt{N_{\text{eff}}}, \qquad N_{\text{eff}} = \frac{N}{1 + (N-1)\,\overline{\rho}}, \tag{13}$$

so every process either raises $\overline{\text{IC}}$ (find real information), raises N_{eff} (kill redundancy), or protects both from estimation error and decay. Nine processes across four layers:

Layer 1 — existence with redundancy control. `S1 cmi_select` (evaluation): greedy forward selection in information space, $f_1 = \arg\max_f I(f;r)$, $f_{k+1} = \arg\max_f I(f;r \mid S_k)$, stopping when the marginal gain $< \max(I_{\min}, \epsilon)$ — uncorrelated-by-construction (catches nonlinear redundancy PCA misses). `S2 interaction_synergy` (evaluation): pairwise synergy $S(X,Y) = I([X,Y];r) - I(X;r) - I(Y;r)$ over the top-K features. `S3 lead_lag_te` (evaluation): cross-symbol directed information $\text{TE}(x_A \to r_B, \text{lag } L)$, edges above $I_{\min}$ with directionality ratio > 1 forming the lead–lag network.

Layer 2 — estimation hygiene. `S4 permutation_null` (meta-process): stationary block bootstrap of returns ($M = 500$), $p_{\text{perm}}(f) = \#\{\text{metric}_{\text{null}} \geq \text{metric}_{\text{obs}}\}/M$, plus White's-Reality-Check excess discoveries. `S5 stability_decay`: rolling calendar-windowed IC with CUSUM / Page–Hinkley break detection and sign-consistency ≥ 0.7.

Layer 3 — conditional structure. `S6 regime_conditional`: conditional IC within regime buckets (lift $= \text{IC}_{\text{active}}/\text{IC}_{\text{uncond}}$) and *stress* correlation of the signal set (per-bucket $\overline{\rho}$ and N_{eff}). `S7 residualize` (transform): $\text{res}_f(t) = f(t) - \beta' Z(t)$ with β fit on the training prefix only. `S8` adds Marchenko–Pastur denoising to `pca_combo`: noise eigenvalues fall below $\lambda_+ = (1 + \sqrt{N/T})^2$; clip the noise band, keep signal eigenvectors.

Layer 4 — combination. `S9 signal_book` (capstone transform): take features that are selected (S1) $\cap$ stable (S5) $\cap$ regime-robust/gated (S6), residualize sequentially into mutually orthogonal `sig_*` columns, and emit an honest manifest with N_{eff} and $\text{IR}_{\text{expected}} = \overline{\text{IC}}\sqrt{N_{\text{eff}} \cdot \text{bars/yr}}$ computed for *both* calm and stress $\overline{\rho}$ (the gap is the risk disclosure). Sequencing (total $\sim$49h): S1+S5+S4 (17h) yield the minimum honest loop; S9 is only worth building on ≥ 7 clean days of data. Every process ships with its planted-signal contract in `synthetic.py` before real-data use.

4.4 MI lineage, target taxonomy, and derivative operators

Why MI (Part A). The lineage runs Shannon (1948) $\rightarrow$ Cover & Thomas (Fano / rate–distortion: MI lower-bounds prediction error, the parent of the `min_info_bits` cost gate) $\rightarrow$ KSG (2004, the estimator) $\rightarrow$ Schreiber (2000) / Granger (1969) for directed information $\rightarrow$ Battiti / Peng–Long–Ding (mRMR) / Brown et al. (2012) for selection $\rightarrow$ Bailey–Borwein–López de Prado–Zhu (2014) for data-snooping correction. The framework already instantiates all four currencies.

Target taxonomy (Part C). "Price action" is a 2-D space of *aspect* $\times$ *horizon*; the codebase currently studies essentially one cell (forward returns / magnitude):

Aspect	Example target	Current status
Direction	sign(r)	binary in `ml_importance` only
Magnitude	$\lvert r\rvert$, r	**shipped** (forward returns)
Volatility	future RV / range	not a target (scaler only) — *high-value gap*
Path	triple-barrier, MFE/MAE, time-to-touch	`tb_label` partial
Regime transition	P(switch)	none
Jump / tail	Lee–Mykland	`jump_detector` algo only
Microstructure	mid $\rightarrow$ micro $\rightarrow$ **fill-conditional**	mid only — *highest-value gap*

The proposed fix (`scripts/processes/targets.py`) promotes the target to a first-class `Target` node (`ForwardReturn`, `Direction`, `RealizedVol`, `TripleBarrier`, `FillConditional`), so *every* process computes I(feature; Target) against one explicit target — closing the gap that `mi_ksg`, `transfer_entropy`, and `spectral` silently use forward returns regardless.

Derivative operators (Part D). A proposed `feature_ops` transform-process (streak-safe, chainable) would mint `op_<operator>_<source>` columns. The highest-value missing operator is **fractional differentiation** $(1 - B)^d$, $0 < d < 1$ (López de Prado AFML Ch.5) — stationarizes a

level series while preserving memory that integer differencing destroys. Others: leaky integration $y_t = \rho y_{t-1} + x_t$; rolling spectral features (bandpower, spectral entropy, wavelet energy); robust normalization (rolling rank/quantile, median/MAD z); cross-feature synergy products; generalized rolling Hurst/DFA; nonlinear bases. Standing caveat: operators multiply the column space, so a program-level FDR ledger matters *more*. The full assembly is `feature_ops` → `cmi_select` → Target → evaluation processes → `signal_book` → algorithm.

5 Machine-Learning Implementation Plan

5.1 Strategy: a gated pipeline, not a waterfall

The plan is a *gated pipeline* with explicit kill criteria at each wave: if ML does not add alpha on this dataset with this much data, work stops early rather than building all ten algorithms on hope. The defining constraint is data — the five deployed winners (`jump_detector`, `3f_liquidity`, `optimal_entry`, `funding_reversion`, `surprise_signal`) are handcrafted and need no training, whereas every ML algorithm here needs training data and only 30 days exist. Algorithm IDs #1–#10 are intrinsic to the upstream spec and do *not* match execution order (which is by wave). Three commitment tiers: **committed** (#5, #1, #4, #2; ∼2600 LOC), **conditional** (#3, #6, #7; ∼950 LOC, gated on Wave 1–2), and **deferred** (#8, #9, #10; not scheduled).

Wave	Algorithms	Gating
0 — Infrastructure prerequisites	(shared infra)	none
1 — Change-point + Momentum (proof of concept)	#5, #1	none
2 — Regime + Mean-reversion + Meta-labeling	#4, #2, #3	Wave 1 Case A/B
3 (conditional) — Regime-LightGBM + KNN	#7, #6	Wave 2 Case A/B
4 (deferred) — HMM, Stacking, Online	#8, #9, #10	not scheduled

5.2 Data sufficiency (hard gates)

Data: 30 days (2026-04-19 to 2026-06-03), 100 ms ticks, ∼29M ticks across three symbols, ∼8640 five-minute bars/symbol. Walk-forward uses 4 folds, 75/25 split, embargo = 100 bars (∼6480 train, ∼2060 effective test bars/fold). Logistic regression (7 features) is safe; LightGBM (31 leaves × 200 trees = 6200 nodes vs 6480 bars) is risky, hence `num_leaves=15`. Four hard thresholds must pass *before* training any model: (1) ≥ 4000 bars/symbol; (2) binary-label positive rate in $[40\%, 60\%]$; (3) per-feature NaN rate $< 5\%$; (4) each test fold ≥ 500 bars (else reduce `n_splits` $4 \to 3$). The verifier computes `n_bars = n_ticks // 3000`. Implications: logistic OK, LightGBM$(15, 100)$ OK, HMM 3–5 states OK, stacking and online *not enough data* — which is exactly why #8/#9/#10 are deferred.

5.3 Wave 0 — infrastructure (1 day)

The keystone is the **bar-level dispatch fix**: add `bar_level=False` and `bar_timeframe="5min"` to the `MicrostructureAlgorithm` ABC (+2 lines in `base.py`), and in `AlgorithmRunner.run_on_dataframe()` (+20 lines) call `aggregate_bars(df, timeframe=algo.bar_timeframe)` when `algo.bar_level` and `timestamp_ns` are present, then forward-fill bar results back to the tick index (`result.reindex(df.index, method='ffill')`). This replaces ∼10 duplicated detection blocks; each ML algorithm just declares

`bar_level=True`. Also: extract `WelfordNormalizer` to `scripts/utils/online.py`; add bar fixtures to `conftest.py` (`make_bar_df`, `make_forward_returns`, `make_labeled_df`, `make_regime_labels` with a 95% self-loop transition matrix); create model directories only for committed algorithms; and run the four data-sufficiency checks.

5.4 Wave 1 — change-point (#5) and momentum (#1) (5 days)

Purpose: answer Q1 (does bar-level integration work end-to-end? — #5, no training) and Q2 (can a trained model produce positive OOS alpha? — #1, the simplest supervised case). If both fail, stop.

#5 ChangePointDetector ($\sim$200 LOC, model-free, `bar_level=True`): CUSUM plus Bayesian online changepoint detection. Parameters `cusum_threshold=5.0`, `cusum_drift=0.05`, `hazard_rate=1/200`, `max_run_length=500`, with a Normal-Inverse-Gamma predictive prior ($\mu_0 = 0$, $\kappa_0 = 1$, $\alpha_0 = 1$, $\beta_0 = 1$). Outputs `alg_cpd_cusum_signal`, `alg_cpd_run_length`, `alg_cpd_change_prob`, `alg_cpd_regime_age`; required columns `imbalance_qty_l1_mean`, `vol_returns_5m_last`, `ent_tick_1m_mean`. Per-bar update:

$$S^+ = \max(0, S^+ + x - \text{drift}), \quad S^- = \max(0, S^- - x - \text{drift}), \tag{14}$$

the Adams–MacKay run-length recursion (growth $= P(r)(1-h)\,\pi$, changepoint $= \sum P(r)\,h\,\pi$, renormalize), expected run length $\sum_r r\,P(r)$, and signal $\max(S^+, S^-) \cdot \text{sign}(S^+ - S^-)$. The run-length vector must be capped at `max_run_length` and renormalized to avoid linear memory growth.

#1 MomentumContinuation ($\sim$250 LOC, logistic regression, `bar_level=True`): the simplest supervised test. Training (`train_momentum.py`): 20-bar forward return fwd$[t]$ = midprice$[t+20]$/midprice$[t]-1$, binary label (assert 40–60% balance), seven bar-aggregated features (`ent_tick_1m_mean`, `ent_permutation_returns_16_mean`, `trend_hurst_300_mean`, `toxic_vpin_50_mean`, `whale_net_flow_4h_sum`, `regime_accumulation_score_mean`, `vol_returns_5m_last`; assert NaN $< 5\%$), `StandardScaler`, `LogisticRegression(C=1.0, l2, max_iter=1000)`, and walk-forward (polars) with `n_splits=4`, `embargo=100`. A LightGBM upgrade (`num_leaves=15`) is attempted *only* if the logistic OOS Sharpe > 0.5. Parameters `entropy_ceiling=0.85`, `p_long=0.6`, `p_short=0.4`; outputs `alg_mc_signal`, `alg_mc_confidence`, `alg_mc_entropy_gate`. `step()` applies an entropy gate (gate = 1 if ent $<$ ceiling else 0) and a dead-zone signal: $(\text{prob} - 0.5) \cdot 2 \cdot \text{gate}$ if prob $> p_{\text{long}}$, $-(0.5 - \text{prob}) \cdot 2 \cdot \text{gate}$ if prob $< p_{\text{short}}$, else 0.

Wave 1 decision gate. **Case A** (#1 OOS Sharpe > 0.5 and 2/3 symbols positive): proceed to Wave 2 in full. **Case B** (> 0.5 but 1/3 symbols): Wave 2 cautious — per-symbol models, skip #3. **Case C** (0.0–0.5): investigate via feature importance before proceeding. **Case D** (< 0.0): stop — ML adds no alpha; try a different horizon/label or accumulate 60+ days. #5 is model-free and survives the gate independently (kept as a regime gate if `alg_cpd_change_prob` vs subsequent volatility $\rho > 0.15$).

5.5 Wave 2 — regime (#4), mean-reversion (#2), meta-labeling (#3) (7 days)

#4 RegimeStateMachine ($\sim$400 LOC): start with *manual thresholds only* (no training) over six states — `ACCUMULATION`, `MARKUP`, `DISTRIBUTION`, `MARKDOWN`, `HIGH_VOLATILITY`, `VOLATILE_NOISE` — from `vol_returns_5m_last`, `trend_hurst_300_mean`, `ent_tick_1m_mean`, `whale_net_flow_4h_sum`,

`toxic_vpin_50_mean`. Its primary value is as a *gate* for other algorithms (notably #7); acceptance requires no state below 5% or above 60% occupancy. **#2 MeanReversionDetector** ($\sim$300 LOC, LightGBM `num_leaves=15`): predicts whether a price extreme reverts within 20 bars; a SHAP step drops any feature with importance < 0.02 and retrains; must be negatively correlated with #1 ($\rho < -0.2$), else redundant. **#3 MetaLabeling** ($\sim$350 LOC, logistic on triple-barrier labels): skipped under Wave-1 Case B. The hard part is preprocessing (`build_meta_training_data`): run all five winners, aggregate their outputs to bars (`alg_jump_signal_mean`, etc.), filter active bars, compute López de Prado triple-barrier labels, and train. Outputs `alg_meta_probability`, `alg_meta_side`, `alg_meta_size`. **Wave 2 gate:** Case A (3+ uncorrelated OOS > 0.5) $\rightarrow$ Wave 3 full; Case B (2 working) $\rightarrow$ Wave 3 partial (#7 only); Case C (1) $\rightarrow$ stop and deploy it; Case D (0) $\rightarrow$ stop.

5.6 Wave 3 (conditional) — regime-LightGBM (#7) and KNN (#6)

#7 RegimeConditionedLightGBM ($\sim$350 LOC): a separate LightGBM per regime plus one global fallback; with $\sim$8640 bars over six regimes (average $\sim$1440), regimes below `min_samples_regime=500` use the fallback. Per-regime feature sets differ (trending: Hurst/whale/accumulation; mean-reverting: entropy/vol/imbalance; volatile: vol/VPIN/entropy). Must beat max(#1, #2) OOS or regime conditioning adds nothing. **#6 KNN retrieval** ($\sim$250 LOC, buffer-based, no training): whiten via the Cholesky of a Ledoit–Wolf covariance and query a KD-tree, refitting the whitening every 100 bars; `K=20`, `buffer_size=5000`, `time_decay_halflife=500`, `min_buffer=100`. Built only under Wave-2 Case A; must satisfy $\rho < 0.4$ with #1/#2.

5.7 Wave 4 (deferred)

#8 (HMM with learned emissions) overlaps #4; #9 (stacking) needs $>$ 15K bars/symbol for a non-noisy out-of-fold protocol; #10 (online SGD) needs a proven, measured concept drift. Re-evaluation triggers: data exceeds 60 days, 4+ uncorrelated ML algorithms deployed, or a deployed model shows $>$ 30% Sharpe degradation over two weeks.

5.8 Cross-cutting: codebase gotchas, verification, and risk

Gotchas (existing-codebase facts). `@register` instantiates `cls()` with *no arguments*, so every `__init__` parameter needs a default (constructor defaults *are* the production values; `config/algorithms.toml` is an optional override and is not auto-injected). `walk_forward_validation()` consumes *polars*, returning `in_sample_sharpe`, `out_of_sample_sharpe`, `oos_is_ratio`, `is_valid`. Bar-aggregation renames columns by suffix (price $\rightarrow$ `_mean`; entropy $\rightarrow$ `_mean/_std/_slope`; flow $\rightarrow$ `_sum`; others $\rightarrow$ `_mean/_std/_last`), so `required_columns()` must use suffixes. `AlgorithmFeature` is frozen (`name`, `warmup` — in bars for bar-level algos — `description`); features need the `alg_` prefix; a registered-but-untrained algorithm returns all-NaN.

Verification matrix. Six phases, advancing only on pass:

Phase	Command	Pass criterion
1. Smoke	`pytest -k <name>`	all 6 `TestSmokeAll`; warmup NaN, rest finite
2. Training (ML)	`python scripts/train_<name>.py`	model saved; `is_valid`; OOS Sharpe > 0.5
3. Feature audit	SHAP / coefficient importance	no feature < 0.02 (else drop + retrain)
4. Single-day	`nat daily`	appears; non-NaN; trades > 0
5. Multi-day	`nat gauntlet run`	net positive; bps/trade > 1.0; 2/3 symbols
6. Complementarity	Spearman correlation	$\lvert\rho\rvert < 0.5$ vs winners, < 0.4 vs ML

The agent submission protocol then applies five gates: IC > 0.10 and dIC > 0.05; net Sharpe > 0.5 after costs; temporal replication (3+ days); symbol replication (2/3); correlation < 0.6 with all promoted signals.

Risk register (highlights). Insufficient data and overfitting are the HIGH risks (mitigated by the $\geq$ 4000-bar gate, `num_leaves=15`, embargo = 100, deflated Sharpe, and killing any OOS/IS ratio < 0.5); data leakage (forward returns at $t + h$, embargo, out-of-fold stacking, automated lookahead test); the sunk-cost fallacy (the wave gates are *hard* gates); plus model staleness (weekly retrain, IC-decay trigger), complexity creep (simplest model first, upgrade only on $> 20\%$ OOS gain), and correlation clustering (keep the better OOS performer when $\rho > 0.5$).

Timeline. One developer: committed work (Wave 0 + Wave 1 + gate) is $\sim$7 days; running Wave 2 brings it to $\sim$15; Wave 3 to $\sim$21. First useful signal at day 6; the most likely outcome is 2–3 algorithms deployed by day 15 with clear evidence for or against expanding. The git workflow is one `feat/ml-wave-N` branch per wave, merged `-no-ff` after the decision-gate metrics are recorded.

6 Cloud Deployment and Operations

6.1 Objective and guardrails

Data continuity is the binding constraint. The cloud box is (a) a redundant, always-on second ingestor independent of su-35, and (b) the deployment vehicle for the wired binary (`[position_tracker]` enabled) while su-35 stays frozen. **Guardrails:** (1) *su-35 zero contact* until the 7-day clean streak completes — the box is a *separate* ingestor; su-35 upgrades only at cutover; (2) *named-service invocations only* — never a bare `docker compose up`; (3) gates imported, never invented; no live capital before G8 + a healthy kill-switch.

6.2 Provisioning and deployment

Sizing: default Hetzner **AX52** dedicated (8-core / 64 GB, $\sim$EUR 60/mo) for ingestion + the evaluation swarm; ingestion-only first on a Hetzner **CX/CPX** VM ($\sim$EUR 10–20/mo, 2 vCPU / 4 GB ample for 3 symbols). Latency: Hetzner EU adds $\sim$250 ms RTT to Hyperliquid (Tokyo) — fine for research ingestion; live execution (T21) needs Tokyo-proximate hosting. Harden (Phase 0): Ubuntu LTS, SSH keys only, `ufw` allow 22/80/443, non-root `nat` user, `loginctl enable-linger nat`, `NAT_HOME=/var/lib/nat`. Secrets/DNS (Phase 1): `.env` with `DOMAIN`, `ACME_EMAIL`, `POSTGRES_PASSWORD`, `GRAFANA_PASSWORD`, `GRAFANA_ANON=false` on a public box, Telegram tokens; A-records for the apex plus `api.`, `dashboard.`, `prometheus.`, `research.`. One-command deploy: `nat deploy cloud <IP> -user nat -nat-home /var/lib/nat` chains scp $\rightarrow$ `apt install` $\rightarrow$ `nat service install` (systemd

`-user` units `nat-ingestor`/`nat-gap-alert`, `Restart=always`, `WantedBy=default.target`) → health check.

6.3 The 15-service Docker stack

The full set (`docker compose config -services`): `redis`, `postgres`, `ingestor`, `gap-alert`, `alerts`, `kill-switch`, `promotion`, `signal-bridge`, `metrics-exporter`, `prometheus`, `grafana`, `caddy`, `api`, `web`, `optuna-dashboard`. Health-gated `depends_on` order: redis → ingestor → gap-alert; kill-switch → signal-bridge; prometheus → grafana; caddy → {grafana, api}; postgres → optuna-dashboard. All services carry `restart: unless-stopped` (auto-restart on crash and host reboot — this, not a script, makes the box 24/7). Bring-up (named services, in order):

```
nat doctor                              # preflight: data-dir ownership, binary, disk
nat docker build
docker compose up -d redis postgres
docker compose up -d ingestor           # WIRED binary; writes ./data/features
docker compose up -d gap-alert alerts kill-switch
docker compose up -d promotion signal-bridge metrics-exporter
docker compose up -d prometheus grafana caddy api web optuna-dashboard
docker compose ps                       # all 15 healthy?
```

6.4 Acceptance: 24h / 48h / 7-day

First 24h — the 40 whale/liquidation/concentration columns leave 100% NaN (per-column non-NaN coverage one-liner), `nat gap status` fresh, and `docker compose logs ingestor | grep WsTrade.users` shows *IS populated + Discovered and promoted whale wallets.* **48h** — apply the concentration viability matrix (§3, Step 05) and record the verdict, which unblocks the LF3 liquidation-cascade work and the agents' dead-column skip lists. **Cutover** — only after the 7-day clean-data streak ($> \sim 200$ MB/day): compare cloud vs su-35 over overlapping hours (row counts, gap profile, feature parity within float noise); the cleaner box becomes primary, su-35 then upgrades to the wired binary.

6.5 Observability

Grafana reads only Prometheus, but NAT's business state lives in SQLite/JSON; the `metrics-exporter` (`scripts/monitoring/metrics_exporter.py`, `:9094/metrics`) bridges them, exporting `nat_lifecycle_signals{` (from `signal_lifecycle` in `nat.db`), `nat_live_cum_pnl_pct` / `nat_live_last_daily_pnl_pct` / `nat_live_pnl_days` (from `data/execution/daily_pnl.json`), and `nat_paper_sharpe{signal}` / `nat_paper_max_drawdown_bps{signal}` (from `data/oos_validation/state.json`). Four auto-provisioned dashboards:

Dashboard	uid	Reads
NAT Lifecycle Funnel	`nat-lifecycle-funnel`	`nat_lifecycle_signals{state}`
NAT Paper Performance	`nat-paper-performance`	`nat_paper_sharpe`, `nat_paper_max_drawdown_bps`
NAT Live P&L	`nat-live-pnl`	`nat_live_*`
NAT Overview	`nat-overview`	ingestor throughput / freshness

The lifecycle funnel highlights `APPROVAL_PENDING` (the human gate). Deferred: per-daemon `/metrics` endpoints and a full `docker compose up` E2E integration test.

6.6 Operations and the silent-stall gotcha

Retention: raw parquet grows per-symbol-per-day — set an expiry/downsample. **Backups:** `data/nat.db` (lifecycle/research) + the postgres volume (Optuna studies). **Data-dir ownership (silent stall):** the Docker ingestor runs as *root* and creates root-owned `data/features` and `data/trades`; a native (non-root) ingestor against the same tree *cannot write and stalls silently* (the writer task dies with no operator error). Fix: `sudo chown -R <user>:<user> data/`; `nat doctor` detects this — run it before switching between Docker and native ingestors.

6.7 Packaging `nat` as a password-gated `.deb`

`nat` ships as a Debian package installable with `sudo apt install nat`, but only on machines holding credentials to a private, GPG-signed apt repository behind HTTP basic-auth. **Build:** `bash packaging/build_deb.sh` (or `nat package deb`) → `dist/nat_<version>_amd64.deb`. **Installed layout:** the CLI at `/usr/lib/nat/nat`, Python under `/usr/lib/nat/scripts/`, Rust binaries (`ing`, `api`, `natviz3d`, validators) under `/usr/lib/nat/rust/target/release/`, config in `/etc/nat/`, symlink `/usr/bin/nat`. Path resolution (`scripts/nat_paths.py`, `nat config paths`) follows precedence `NAT_HOME`/`NAT_*` env → source checkout → installed XDG. **Durable supervision:** `nat service install` writes systemd `-user` units (auto-restart + boot-persistence via linger), superseding the default tmux+cron watchdogs that die on reboot. **Publish:** sign with a dedicated GPG key, build a flat repo with `aptly`, serve `~/.aptly/public` over HTTPS basic-auth (Caddy). **Onboard a client:** `scripts/setup_apt_client.sh` writes the keyring, `sources.list.d/nat.list`, and a mode-600 `auth.conf.d/nat.conf`; without credentials `apt update` returns 401, and metadata is GPG-verified so an attacker can neither install nor tamper.